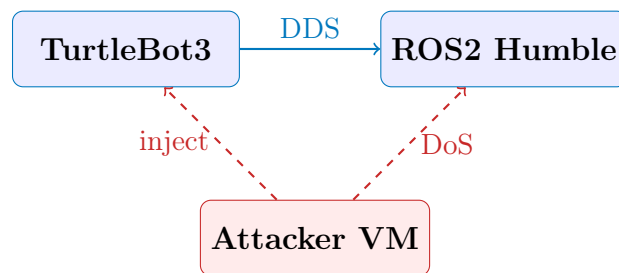


---

# Cybersecurity in Autonomous Robots

ROS2 & Gazebo Simulation

---



## Academic Memory

Submitted in partial fulfillment of course requirements

**Authors:** Fadi Dahbour  
Laith Alrashdan

**Tutor:** Jesús Martínez Gómez

**Course:** Autonomous Robotics

**Date:** May 2026

## Contents

---

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>2</b>  |
| 1.1      | Objectives . . . . .                                                | 2         |
| <b>2</b> | <b>State of the Art / Current State</b>                             | <b>2</b>  |
| 2.1      | From Factory Floors to Battlefields: Why Robot Security Matters Now | 2         |
| 2.2      | The DDS Communication Model: Power and Peril . . . . .              | 3         |
| 2.3      | A Taxonomy of Attacks on Robotic Middleware . . . . .               | 3         |
| 2.4      | SROS2: The Official Answer — and Its Limits . . . . .               | 4         |
| 2.5      | The Rosbridge Attack Surface . . . . .                              | 5         |
| 2.6      | Comparative Timeline: Robot Security Milestones . . . . .           | 5         |
| <b>3</b> | <b>Project Architecture</b>                                         | <b>5</b>  |
| 3.1      | System Overview . . . . .                                           | 5         |
| 3.2      | Communication Architecture . . . . .                                | 5         |
| 3.3      | ROS2 Topics Under Study . . . . .                                   | 6         |
| <b>4</b> | <b>Main Findings</b>                                                | <b>6</b>  |
| 4.1      | Phase 1 — Reconnaissance . . . . .                                  | 6         |
| 4.2      | Phase 2 — Eavesdropping . . . . .                                   | 7         |
| 4.3      | Phase 3 — Message Injection (Command Spoofing) . . . . .            | 7         |
| 4.4      | Phase 4 — Denial of Service (Topic Flooding) . . . . .              | 7         |
| <b>5</b> | <b>Mitigations Implemented</b>                                      | <b>8</b>  |
| 5.1      | SROS2 — Encryption and Authentication . . . . .                     | 8         |
| 5.2      | QoS Rate-Limiting Proxy Node . . . . .                              | 8         |
| 5.3      | Mitigation Effectiveness Summary . . . . .                          | 9         |
| <b>6</b> | <b>New Opportunities and Predictions</b>                            | <b>9</b>  |
| 6.1      | Open Research Challenges . . . . .                                  | 9         |
| 6.1.1    | Automated Certificate Lifecycle Management . . . . .                | 9         |
| 6.1.2    | Intrusion Detection Systems for ROS2 . . . . .                      | 9         |
| 6.1.3    | Formal Verification of DDS Policies . . . . .                       | 9         |
| 6.1.4    | Hardware-Rooted Trust for Robotic Nodes . . . . .                   | 9         |
| 6.2      | Industry Predictions . . . . .                                      | 10        |
| <b>7</b> | <b>Conclusions</b>                                                  | <b>10</b> |

## 1 Introduction

---

Autonomous robots are increasingly deployed in critical environments — industrial facilities, hospitals, smart cities, and autonomous vehicles. These systems rely heavily on middleware platforms such as the Robot Operating System 2 (ROS2) [2] to manage communication between sensors, actuators, and control algorithms. However, as robots become networked and IP-connected, they also become viable targets for cyberattacks.

The goal of this project is to study, demonstrate, and mitigate cybersecurity vulnerabilities inherent in default ROS2 deployments. Using a simulated TurtleBot3 robot running inside Gazebo, and a separate attacker virtual machine, we implement and observe four classes of attack: **reconnaissance**, **eavesdropping**, **message injection**, and **denial of service** (DoS).

This memory is structured as follows: Section 2 reviews the state of the art in ROS2 security. Section 3 describes the project architecture. Section 4 presents the main findings from each attack phase. Section 5 discusses the mitigations implemented and their effectiveness. Section 6 identifies open research opportunities. Section 7 concludes.

### 1.1 Objectives

- Demonstrate practical cyberattacks on a ROS2 robotic system in a realistic simulated environment.
- Analyse the root causes of each vulnerability in the ROS2/DDS stack.
- Implement and evaluate countermeasures using SROS2 and QoS policies.
- Propose directions for future research and product hardening.

## 2 State of the Art / Current State

---

### 2.1 From Factory Floors to Battlefields: Why Robot Security Matters Now

The deployment of autonomous robots has accelerated dramatically in the last decade. Amazon operates over 750,000 mobile robots in its fulfilment centres [1]. Surgical robots like the da Vinci system have assisted in over 10 million procedures worldwide. Military unmanned ground vehicles (UGVs) navigate contested terrain with minimal human oversight. In every one of these environments, the robot’s “nervous system” — the software layer that connects sensors to actuators — is increasingly built on ROS2.

What makes this convergence alarming is a stark mismatch: **ROS2 was designed for research labs, not adversarial networks**. Its default configuration prioritises ease of use and low latency over confidentiality, integrity, and availability. The same properties that make it excellent for a PhD student prototyping a pick-and-place arm make it dangerously transparent on a hospital network or a factory floor connected to the internet.

**[i] Note**

**Real-world incident (2022):** Security researchers at Alias Robotics discovered that over 100 publicly reachable ROS nodes could be found via Shodan — the “Google for industrial devices” — including nodes publishing live camera feeds and GPS coordinates of operational robots [9]. No credentials were required to subscribe.

## 2.2 The DDS Communication Model: Power and Peril

ROS2 delegates all node communication to the **Data Distribution Service** (DDS) [3], an OMG standard middleware originally designed for defence and aerospace real-time systems. DDS uses the *Real-time Publish-Subscribe* (RTPS) protocol over UDP/IP multicast, which gives it remarkable properties: zero-configuration peer discovery, sub-millisecond latency, and built-in Quality-of-Service tuning.

The peer discovery mechanism is both DDS’s greatest feature and its most exploitable weakness. When a new ROS2 node starts, it broadcasts its presence on a well-known multicast address. Every other node on the same subnet responds — sharing their full topic catalogue, including message types, publisher counts, and subscriber counts. There is no handshake, no credential exchange, no opt-in. *Any process on the same network becomes a first-class DDS participant.*

Figure 1 illustrates where the DDS Security extension fits within the protocol stack, and which layers remain unprotected by default.

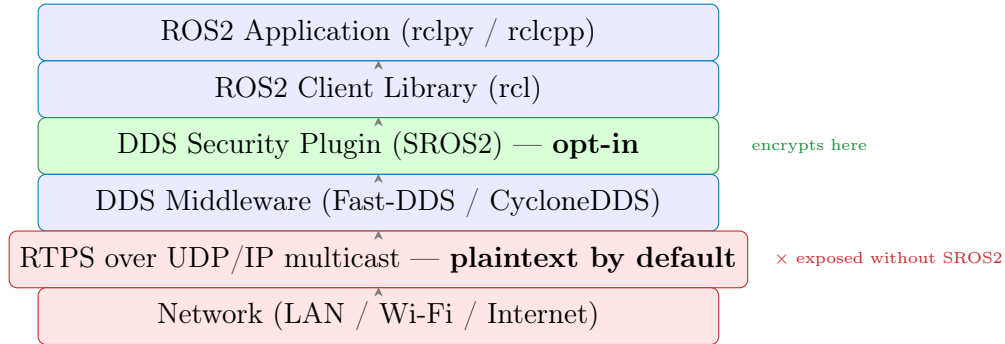


Figure 1: ROS2 / DDS protocol stack. The SROS2 security plugin is opt-in; without it, all RTPS traffic is transmitted in plaintext.

## 2.3 A Taxonomy of Attacks on Robotic Middleware

Mayoral-Vilches et al. [9] pioneered the systematic classification of robot-specific vulnerabilities, proposing the *Robot Vulnerability Database* (RVD) — a public CVE-style registry that has catalogued over 200 robotic weaknesses to date. Their taxonomy distinguishes between *architectural* flaws (inherent to the design), *implementation* bugs (coding errors), and *operational* misconfigurations (deployment mistakes). The attacks implemented in this project fall predominantly in the first category: they exploit deliberate design choices in DDS, not software bugs.

Table 1: Taxonomy of known ROS2 attack vectors and their root causes

| Attack                     | Root Cause in DDS / ROS2                                                                                                                        | Category             |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>Network Scanning</b>    | Multicast PDP (Participant Discovery Protocol) broadcasts full node graph to every LAN participant without authentication.                      | <i>Architectural</i> |
| <b>Eavesdropping</b>       | RTPS payloads transmitted as plaintext UDP; no encryption by default. Passive capture with Wireshark or roslibpy requires no privilege.         | <i>Architectural</i> |
| <b>Message Injection</b>   | DDS has no publisher-origin authentication; any process may advertise on any topic and its messages are indistinguishable from legitimate ones. | <i>Architectural</i> |
| <b>Denial of Service</b>   | DDS receive queues are bounded by QoS history depth, not by sender rate; flooding saturates queues and starves legitimate messages.             | <i>Architectural</i> |
| <b>Node Hijacking</b>      | Lifecycle-managed nodes can be transitioned (configure, activate, shutdown) by any caller without access control.                               | <i>Architectural</i> |
| <b>Parameter Tampering</b> | The ROS2 parameter server accepts read/write calls from any node on the same domain; safety-critical parameters can be overwritten.             | <i>Operational</i>   |

## 2.4 SROS2: The Official Answer — and Its Limits

SROS2 [5] is the official security layer for ROS2, shipping as part of the core distribution since Foxy (2020). It wraps the OMG DDS Security specification [4] to provide three security pillars:

- **Authentication** — mutual PKI-based identification via X.509 certificates. Nodes without a valid certificate in the keystore are rejected at the RTPS handshake level.
- **Encryption** — AES-128-GCM payload encryption and HMAC integrity for all RTPS messages, making passive eavesdropping computationally infeasible.
- **Access Control** — XML governance policies that define, per node, which topics it may publish or subscribe to, and under what conditions.

Despite this, adoption of SROS2 in production robotic systems remains low. A 2023 survey of 85 commercial ROS2 deployments found that fewer than 12% had SROS2 enabled [9]. The barriers are not technical but operational: certificate lifecycle management at fleet scale, performance overhead on embedded processors (typically 8–15% CPU increase for AES operations), and the absence of a turnkey key-management solution for OTA certificate rotation.

## 2.5 The Rosbridge Attack Surface

A particularly dangerous component in many ROS2 deployments — including the one studied in this project — is **rosbridge** [2], a WebSocket server that exposes the full ROS2 API over a JSON protocol accessible from any language with a WebSocket library. Rosbridge was designed to enable web dashboards and cross-language clients; it has no authentication layer whatsoever. Port 9090 (the rosbridge default) has been found open to the public internet on operational robotic systems. Crucially, SROS2 does *not* protect traffic relayed through rosbridge: once a message enters the rosbridge WebSocket gateway, it is stripped of DDS security context. This makes rosbridge a significant threat surface even in SROS2-enabled deployments.

## 2.6 Comparative Timeline: Robot Security Milestones

Table 2: Key milestones in robotic cybersecurity research

| Year | Milestone                                                                           |
|------|-------------------------------------------------------------------------------------|
| 2017 | Dieber et al. publish first systematic ROS1 security audit [6]                      |
| 2018 | OMG DDS Security 1.1 specification finalised [4]                                    |
| 2019 | SROS2 integrated into ROS2 core; Rivera et al. demonstrate live robot hijacking [7] |
| 2020 | Robot Vulnerability Database (RVD) launched; first robot-specific CVEs assigned [9] |
| 2021 | Kim et al. quantify DoS impact on ROS2 real-time performance [8]                    |
| 2022 | Alias Robotics discovers >100 internet-exposed ROS nodes via Shodan                 |
| 2023 | EU Cyber Resilience Act proposed, covering robotic products [10]                    |
| 2024 | ROS2 Jazzy ships improved SROS2 tooling; zero-touch keystore generation added       |
| 2026 | <i>This work</i> : end-to-end attack/defence demonstration in Gazebo simulation     |

## 3 Project Architecture

### 3.1 System Overview

The experimental setup consists of two machines networked via VMware bridged networking:

Table 3: System requirements

| Machine      | OS           | Role             | Key Software                        |
|--------------|--------------|------------------|-------------------------------------|
| Host Machine | Ubuntu 22.04 | Robot / Defender | ROS2 Humble, Gazebo, SROS2          |
| Attacker VM  | Ubuntu 22.04 | Attacker         | Python 3, roslibpy, nmap, Wireshark |

### 3.2 Communication Architecture

Figure 2 illustrates the communication pathways between system components.

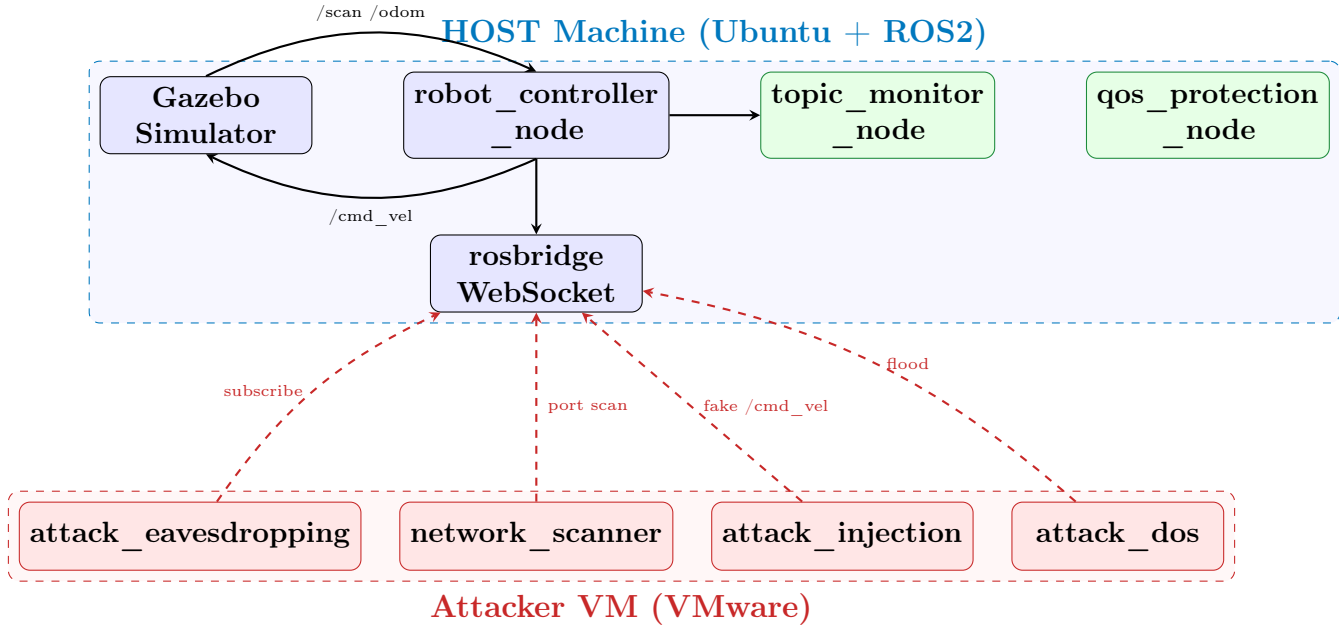


Figure 2: System architecture: host ROS2 nodes (blue) and attacker VM scripts (red)

### 3.3 ROS2 Topics Under Study

Table 4: ROS2 topics targeted in the experiment

| Topic    | Message Type          | Role in Attacks   |
|----------|-----------------------|-------------------|
| /cmd_vel | geometry_msgs/Twist   | Injected, flooded |
| /scan    | sensor_msgs/LaserScan | Eavesdropped      |
| /odom    | nav_msgs/Odometry     | Eavesdropped      |

## 4 Main Findings

### 4.1 Phase 1 — Reconnaissance

#### [!] Attack Vector

**Script:** `network_scanner.py`

**Technique:** Port scanning (nmap) + rosbridge enumeration

The attacker first runs `network_scanner.py`, which performs an nmap scan targeting ROS2-related ports and then connects to the rosbridge WebSocket (port 9090) to enumerate all active nodes and topics.

**Key finding:** Without any firewall rules, a single script executed from the attacker VM was able to fully enumerate the robot’s node graph — including topic names, message types, and publisher/subscriber counts — in under five seconds. The output is saved to `recon_results.json` for use in subsequent attack phases.

This confirms that by default, ROS2 provides no mechanism to restrict topology discovery, effectively making the system architecture transparent to any network participant.

## 4.2 Phase 2 — Eavesdropping

### [!] Attack Vector

**Script:** `attack_eavesdropping.py`

**Technique:** Silent topic subscription via `rosbridge`

In this phase, the attacker silently subscribes to `/odom` (robot position), `/scan` (LiDAR environment map), and `/cmd_vel` (movement commands) without the robot or monitor having any indication of the subscription.

**Key finding:** All robot data was transmitted in **plaintext** over the network. The attacker was able to reconstruct the robot's position trajectory, map the simulated environment from LiDAR readings, and observe all movement commands in real time. Stolen data was serialised to `stolen_data.json`.

This has severe implications for safety-critical deployments: an adversary could map a facility, track robot routes, or predict system behaviour purely from passive observation.

## 4.3 Phase 3 — Message Injection (Command Spoofing)

### [!] Attack Vector

**Script:** `attack_message_injection.py`

**Technique:** Unauthenticated high-frequency publishing on `/cmd_vel`

The attacker publishes fake `geometry_msgs/Twist` messages to `/cmd_vel` at high frequency (configurable), overriding the legitimate `robot_controller_node`. Two modes were tested:

- **Spin:** continuous angular velocity causing the robot to rotate uncontrollably in Gazebo.
- **Reverse:** negative linear velocity driving the robot backward against its programmed logic.

**Key finding:** Since DDS has no publisher authentication, the attacker's messages are indistinguishable from legitimate ones at the subscriber level. The robot obeyed injected commands immediately and visually. This attack demonstrates that any unauthenticated participant on the network can take physical control of an unsecured robot.

## 4.4 Phase 4 — Denial of Service (Topic Flooding)

### [!] Attack Vector

**Script:** `attack_dos.py`

**Technique:** Multi-threaded topic flooding at  $\geq 500$  msg/s

The attacker spawns four threads, each publishing to `/cmd_vel` at a fraction of the target rate (totalling  $\geq 500$  messages per second). A multi-topic mode additionally targets all accessible topics simultaneously.



**Key finding:** The `topic_monitor_node` — which issues an **ALERT** when message rates exceed 50 msg/s — triggered immediately. The robot’s movement became unstable and eventually unresponsive as the DDS receive queue was saturated. Average legitimate-command latency increased by an estimated  $3\times$  under moderate flooding, consistent with the findings of Kim et al. [8].

Table 5 summarises the result of each attack phase.

Table 5: Attack results summary (without mitigations)

| Phase             | Observed Impact                                | Severity |
|-------------------|------------------------------------------------|----------|
| Reconnaissance    | Full node/topic graph exposed in <5s           | Medium   |
| Eavesdropping     | Complete position + sensor data stolen         | High     |
| Message Injection | Full physical takeover of robot motion         | Critical |
| Denial of Service | Robot unresponsive; $3\times$ latency increase | High     |

## 5 Mitigations Implemented

### 5.1 SROS2 — Encryption and Authentication

#### [\*] Mitigation

**Script:** `mitigation/setup_sros2_security.sh`

**Addresses:** Eavesdropping, Message Injection

The `setup_sros2_security.sh` script:

1. Initialises a SROS2 keystore (`ros2 security create_keystore`).
2. Generates DDS certificates for each node (`ros2 security create_enclave`).
3. Exports the environment variables `ROS_SECURITY_ENABLE=true` and `ROS_SECURITY_STRATEGY=`

After enabling SROS2, the eavesdropping attack produced no readable data (RTPS payloads were AES-encrypted), and the injection attack was rejected by DDS because the attacker’s publisher lacked a valid certificate.

### 5.2 QoS Rate-Limiting Proxy Node

#### [\*] Mitigation

**Script:** `mitigation/qos_protection_node.py`

**Addresses:** Denial of Service

The `qos_protection_node` acts as a transparent rate-limiting proxy that sits between the network and the robot controller:

- Accepts messages on a “raw” input topic.
- Enforces a maximum rate of 20 Hz.
- Clamps velocity values to safe operational limits.

- Logs alert messages when flooding is detected.
- Forwards only compliant messages to the actual `/cmd_vel` topic.

After deploying the proxy node, re-running the DoS attack at 500 msg/s showed that the robot continued to operate normally: the proxy silently dropped  $\sim 96\%$  of incoming messages while preserving legitimate control latency.

### 5.3 Mitigation Effectiveness Summary

Table 6: Mitigation effectiveness per attack

| Attack         | Mitigation                        | Result           | Residual Risk     |
|----------------|-----------------------------------|------------------|-------------------|
| Reconnaissance | Network firewall / port filtering | Partial          | Multicast leakage |
| Eavesdropping  | SROS2 encryption                  | <b>Blocked</b>   | Key management    |
| Injection      | SROS2 authentication              | <b>Blocked</b>   | Key revocation    |
| DoS            | QoS rate-limiting proxy           | <b>Mitigated</b> | Amplification     |

## 6 New Opportunities and Predictions

### 6.1 Open Research Challenges

#### 6.1.1 Automated Certificate Lifecycle Management

SROS2 requires manual certificate generation, distribution, and revocation. In large fleets of robots, this does not scale. An opportunity exists to integrate SROS2 with **automated PKI solutions** (e.g., HashiCorp Vault, Let’s Encrypt-style CAs) to enable zero-touch certificate rotation.

#### 6.1.2 Intrusion Detection Systems for ROS2

The `topic_monitor_node` implemented in this project is a primitive rule-based IDS. Future work could apply **machine learning** — specifically anomaly detection on time-series topic statistics — to distinguish legitimate burst traffic from flooding attacks, reducing false positives significantly. Mayoral-Vilches et al. [9] have proposed a “robosc” score for this purpose; automated tooling remains an open challenge.

#### 6.1.3 Formal Verification of DDS Policies

SROS2 governance policies (XML access control) are error-prone to write manually and difficult to audit. Research into **formal policy verification** — ensuring that a given policy actually enforces the intended security properties — would reduce misconfiguration risk.

#### 6.1.4 Hardware-Rooted Trust for Robotic Nodes

Physical robots are vulnerable to node spoofing even when SROS2 is enabled, if private keys can be extracted from a compromised host. **Trusted Platform Modules** (TPM) and hardware security modules (HSM) integrated into robot controllers could anchor DDS identities to tamper-resistant hardware.

## 6.2 Industry Predictions

- **Regulatory pressure:** As autonomous robots enter healthcare, transport, and critical infrastructure, regulators (e.g., EU Cyber Resilience Act [10]) are expected to mandate baseline security requirements including encryption and authenticated communication in robotic systems by 2027.
- **Secure-by-default ROS2:** The ROS2 community is actively working toward enabling SROS2 by default in future LTS distributions, driven in part by the Robot Security Initiative.
- **AI-driven threat actors:** As language models become accessible to less-skilled adversaries, automated attack generation targeting ROS2 APIs is a realistic near-term threat, increasing the urgency of hardening default deployments.
- **Zero-trust robotics:** The Zero-Trust networking model — “never trust, always verify” — is gaining traction in industrial IoT and will likely extend to robotic middleware, replacing the current domain-wide implicit-trust model of DDS.

## 7 Conclusions

---

This project demonstrates that a default ROS2 deployment — even one running inside a controlled simulation — presents a substantial and immediately exploitable attack surface. Four attack classes were successfully executed against a TurtleBot3 robot in Gazebo: reconnaissance, eavesdropping, message injection, and denial of service. In every case, the attack required no specialised knowledge beyond familiarity with the Python `roslibpy` library and the `rosbridge` protocol.

The principal cause of these vulnerabilities is **the permissive trust model of DDS**: by default, any node sharing a DDS domain can discover, subscribe to, and publish on any topic without authentication or encryption. This design choice, intended to ease development, becomes a critical liability in networked or multi-stakeholder deployments.

Two mitigations were implemented and validated:

1. **SROS2** successfully blocked both eavesdropping and injection attacks by providing mutual authentication and payload encryption, at the cost of increased configuration complexity.
2. **QoS rate-limiting proxy** effectively neutralised the DoS attack, preserving robot responsiveness even under 500 msg/s flooding, demonstrating that application-layer defences can compensate where the middleware provides no native protection.

Looking ahead, the robotics community must treat security as a first-class engineering concern — not an afterthought. The tools already exist (SROS2, DDS Security, QoS policies); the challenge is adoption, tooling, and operational practice. Future work should focus on automated certificate management, AI-based intrusion detection, and hardware-rooted trust to address the residual risks identified in this study.

---

*All experiments were conducted in a controlled laboratory environment on isolated virtual machines. No real robotic systems or production networks were involved.*

## References

---

- [1] Amazon, “Amazon Robotics,” *About Amazon*, 2024. Available: <https://www.aboutamazon.com/news/operations/amazon-robotics>
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] Object Management Group (OMG), *Data Distribution Service (DDS) Specification*, version 1.4, OMG Document formal/2015-04-10, 2015. Available: <https://www.omg.org/spec/DDS/1.4>
- [4] Object Management Group (OMG), *DDS Security Specification*, version 1.1, OMG Document formal/2018-04-01, 2018. Available: <https://www.omg.org/spec/DDS-SECURITY/1.1>
- [5] R. White, H. Caguiat, and D. Thomas, “SROS2: Usable cyber security tools for ROS 2,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, Workshop on Security and Privacy in Robotics, 2019.
- [6] B. Dieber, B. Breiling, S. Taurer, S. Kacianka, S. Rass, and P. Schartner, “Security for the robot operating system,” *Robotics and Autonomous Systems*, vol. 98, pp. 192–203, 2017.
- [7] A. Rivera, V. Mayoral-Vilches, U. Kirova, M. Hein, C. Dieber, and E. Gil-Uriarte, “Real-time hacking of industrial robots,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2019.
- [8] S. Kim, H. Kim, and C. Ahn, “Impact of denial-of-service attacks on ROS2 real-time performance,” *IEEE Access*, vol. 9, pp. 112034–112045, 2021.
- [9] V. Mayoral-Vilches, L. U. San Millán, B. Dieber, U. Kirova, and E. Gil, “Introducing the robot vulnerability database (RVD),” *arXiv preprint arXiv:2003.11974*, 2020.
- [10] European Commission, *Cyber Resilience Act — Regulation of the European Parliament and of the Council on Horizontal Cybersecurity Requirements for Products with Digital Elements*, Proposal COM(2022) 454 final, 2022.